UNIVERSITY OF ILLINOIS URBANA-CHAMPAIGN

ECE408 - CS483 - CSE408 Spring 2026
Applied Parallel Programming

# Lecture 8: Convolution Concept; Constant Cache

Volodymyr Kindratenko

2/12/2026

# Course Reminders

- Lab updates
  - Lab 3 is due this week on **Friday**
- Project README files are released
  - and so is the signup form for GPT-2 project
- Take a note of Midterm 1 date/time
  - **When:** March 3rd, 7-10pm
  - **Where:** Your specific room assignment will be posted in Canvas
  - **What:** Lectures 1-12, Labs 1-4
  - **How:** paper-based
  - **Alternative Exam Time:** as arranged, email me by 2/24/26 if you have a valid conflict
  - **Study materials:** will be posted on Canvas

# Today's Objectives

- To learn convolution, an important parallel computation pattern
  - Widely used in signal, image and video processing
  - Foundational to stencil computation used in many science and engineering applications
  - Critical component of Neural Networks and Deep Learning

- Important GPU technique
  - Taking advantage of cache memories

# Convolution Applications

- A popular operation that is used in various forms in signal processing, digital recording, image processing, video processing, computer vision, and machine learning.

- Convolution is often performed as a **filter** that transforms the input signal (audio, video, etc.) in some context-aware way.
  - Some filters smooth out the signal values so that one can see the big-picture trend

# Convolution Mathematics

$$f(x) * g(x) = \int_{-\infty}^{\infty} f(\tau) \cdot g(x-\tau)\, d\tau$$

$$f[x] * g[x] = \sum_{k=-\infty}^{\infty} f[k] \cdot g[x-k]$$

# Convolution vs. Cross-Correlation

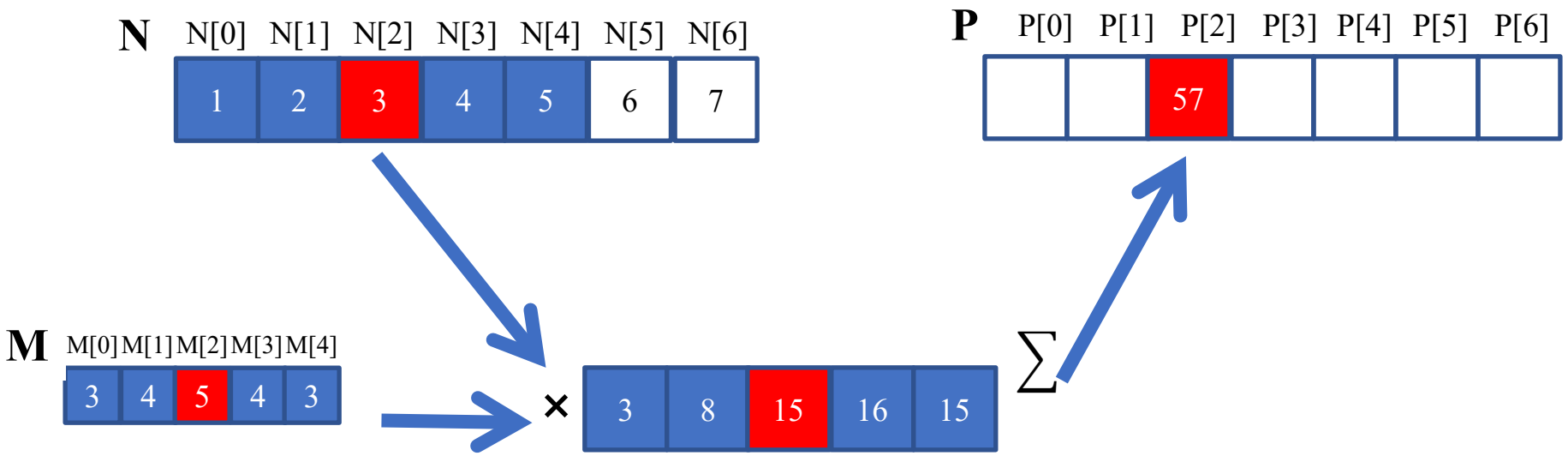
- Convolution
  - $(f*g)(t)=\int f(\tau)g(t-\tau)d\tau$
  - Kernel is flipped
  - Main use: physics, filtering
- Cross-correlation
  - $(f\star g)(t)=\int f(\tau)g(t+\tau)d\tau$
  - No flipping of the kernel
  - Main use: pattern matching, ML
- Convolutional Neural Networks (CNNs) use cross-correlation, not true convolution
  - Whether the kernel is flipped or not makes no difference - the network will learn the appropriate weights anyway.
  - Implementing convolution without flipping (i.e., cross-correlation) is simpler and computationally cleaner.
- Even though CNNs use cross-correlation, the term “convolution” stuck historically.

# Convolution Computation

- An array operation where each output data element is a weighted sum of a collection of neighboring input elements
- The weights used in the weighted sum calculation are defined by an input mask array, commonly referred to as the *convolution kernel*
  - We will refer to these mask arrays as convolution masks or convolution filters to avoid confusion.
  - The same convolution mask is typically used for all elements of the array.

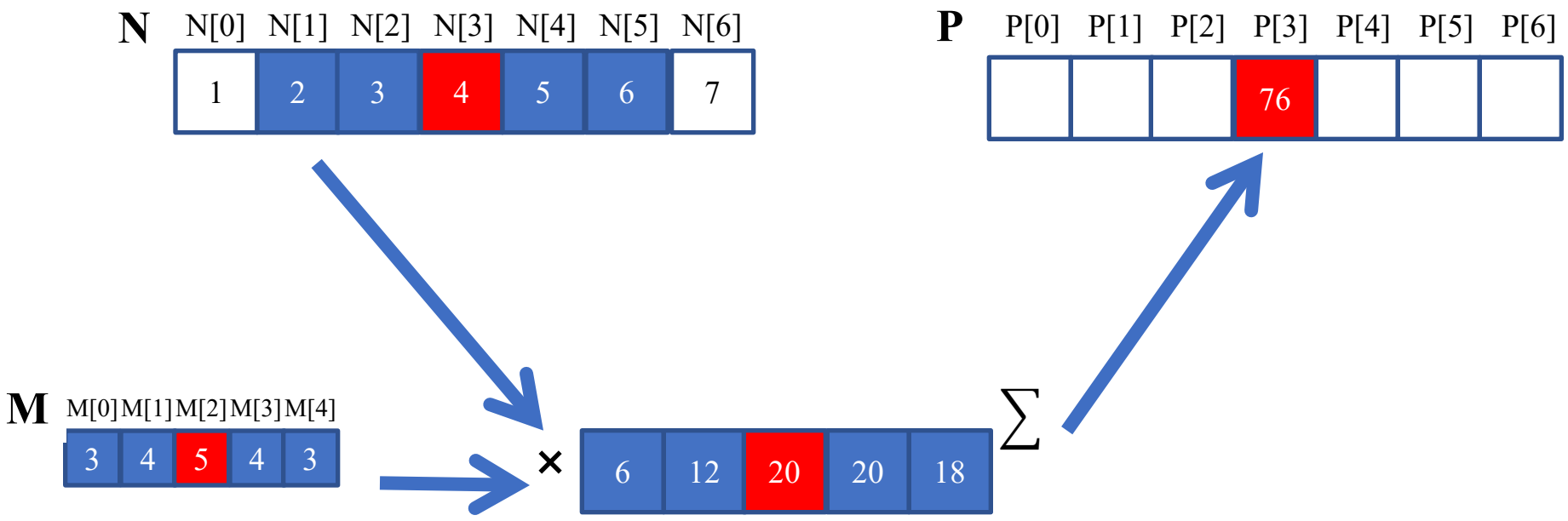
# 1D Convolution Example

- Commonly used for audio processing
  - MASK_WIDTH is usually an odd number of elements for symmetry (5 in this example)
  - MASK_RADIUS is the number of elements used in convolution on each side of the pixel being calculated (2 in this example).
- Calculation of P[2]:



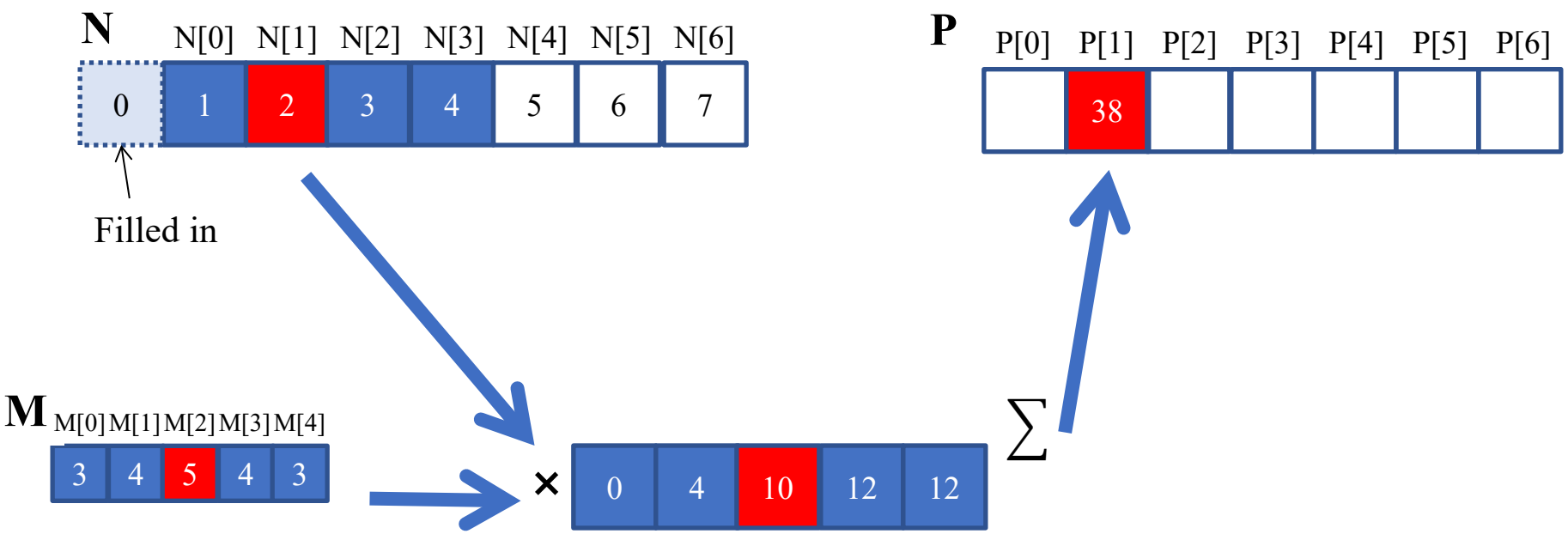
# 1D Convolution Example

- Calculation of P[3]:



# 1D Convolution Boundaries

- Calculation of output elements near the boundaries of the input array need to deal with "ghost" elements
  - Different policies (0, replicates of boundary values, etc.)



# A 1D Convolution Kernel with Boundary Handling

- This kernel forces all elements outside the valid range to 0

```
__global__ void
convolution_1D_basic_kernel(float *N, float *M, float *P, int Mask_Width, int Width)
{
  int i = blockIdx.x*blockDim.x + threadIdx.x;

  float Pvalue = 0;
  int N_start_point = i - (Mask_Width/2);

  for (int j = 0; j < Mask_Width; j++) {
    if (((N_start_point + j) >= 0) && ((N_start_point + j) < Width)) {
      Pvalue += N[N_start_point + j] * M[j];
    }
  }

  P[i] = Pvalue;
}
```

# 2D Convolution

**N**

| | | | | | | |
|---|---|---|---|---|---|---|
| 1 | 2 | 3 | 4 | 5 | 6 | 7 |
| 2 | 3 | 4 | 5 | 6 | 7 | 8 |
| 3 | 4 | 5 | 6 | 7 | 8 | 9 |
| 4 | 5 | 6 | 7 | 8 | 5 | 6 |
| 5 | 6 | 7 | 8 | 5 | 6 | 7 |
| 6 | 7 | 8 | 9 | 0 | 1 | 2 |
| 7 | 8 | 9 | 0 | 1 | 2 | 3 |

**P**

| | | | | | | |
|---|---|---|---|---|---|---|
| | | | | | | |
| | | | | | | |
| | | 321 | | | | |
| | | | | | | |
| | | | | | | |
| | | | | | | |
| | | | | | | |

**M**

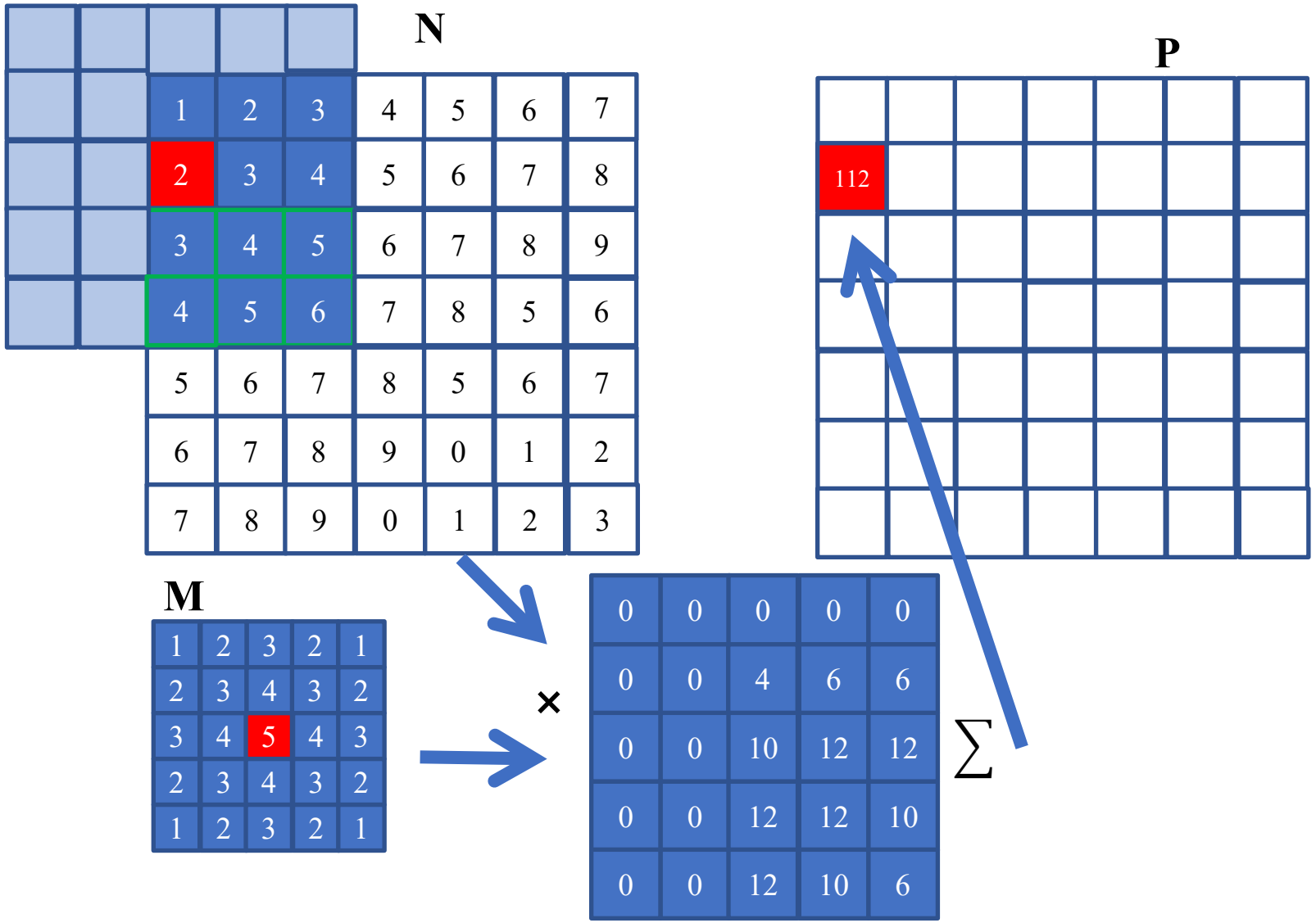
| | | | | |
|---|---|---|---|---|
| 1 | 2 | 3 | 2 | 1 |
| 2 | 3 | 4 | 3 | 2 |
| 3 | 4 | 5 | 4 | 3 |
| 2 | 3 | 4 | 3 | 2 |
| 1 | 2 | 3 | 2 | 1 |

×

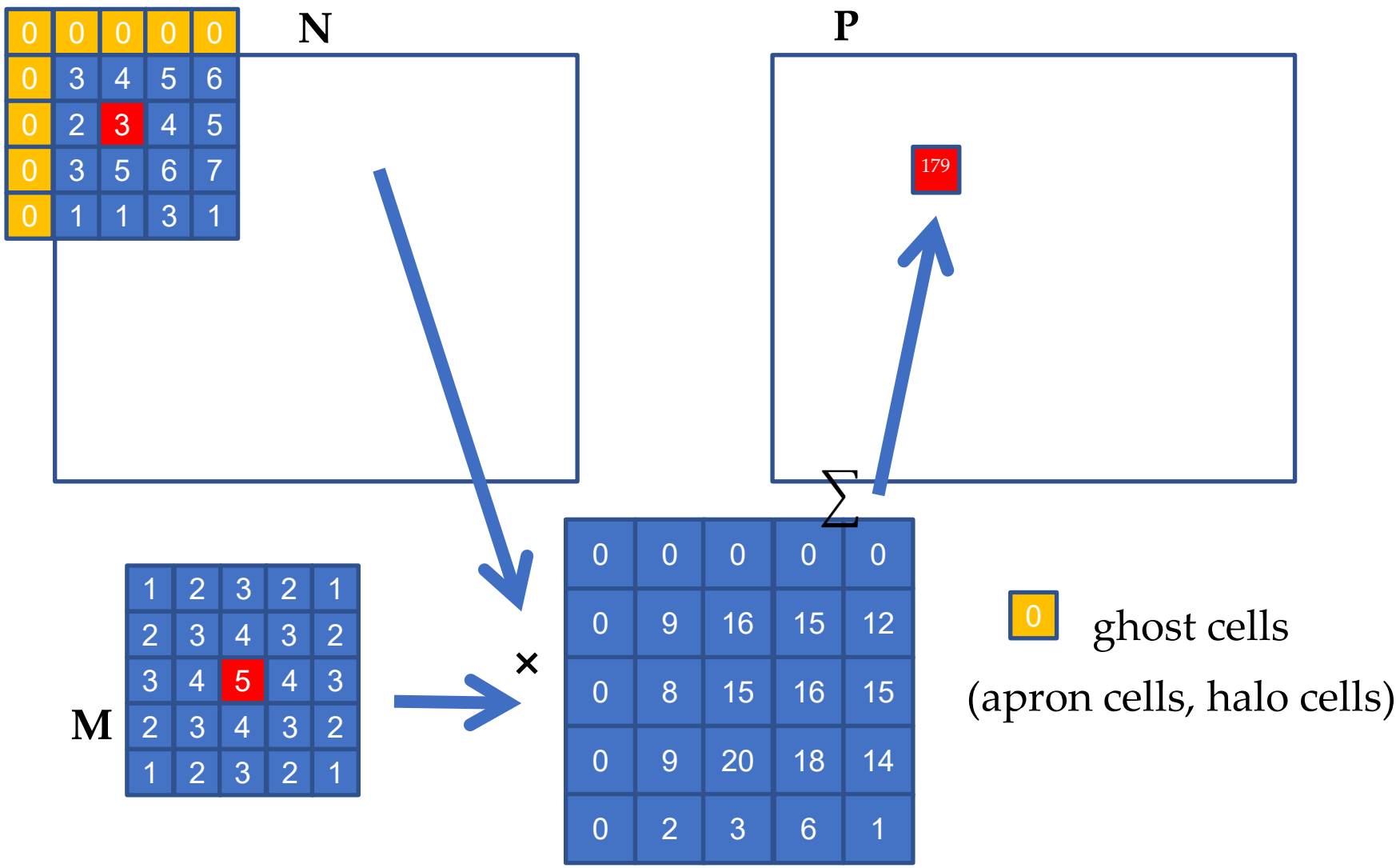
| | | | | |
|---|---|---|---|---|
| 1 | 4 | 9 | 8 | 5 |
| 4 | 9 | 16 | 15 | 12 |
| 9 | 16 | 25 | 24 | 21 |
| 8 | 15 | 24 | 21 | 16 |
| 5 | 12 | 21 | 16 | 5 |

$\sum$

# 2D Convolution Boundary Condition



# 2D Convolution – Ghost Cells



# Example: What does this kernel accomplish?

$$\mathbf{M} = \frac{1}{273} \times \begin{bmatrix} 1 & 4 & 7 & 4 & 1 \\ 4 & 16 & 26 & 16 & 4 \\ 7 & 26 & 41 & 26 & 7 \\ 4 & 16 & 26 & 16 & 4 \\ 1 & 4 & 7 & 4 & 1 \end{bmatrix}$$

Hint: Assume input N is an image

# Access Pattern for M

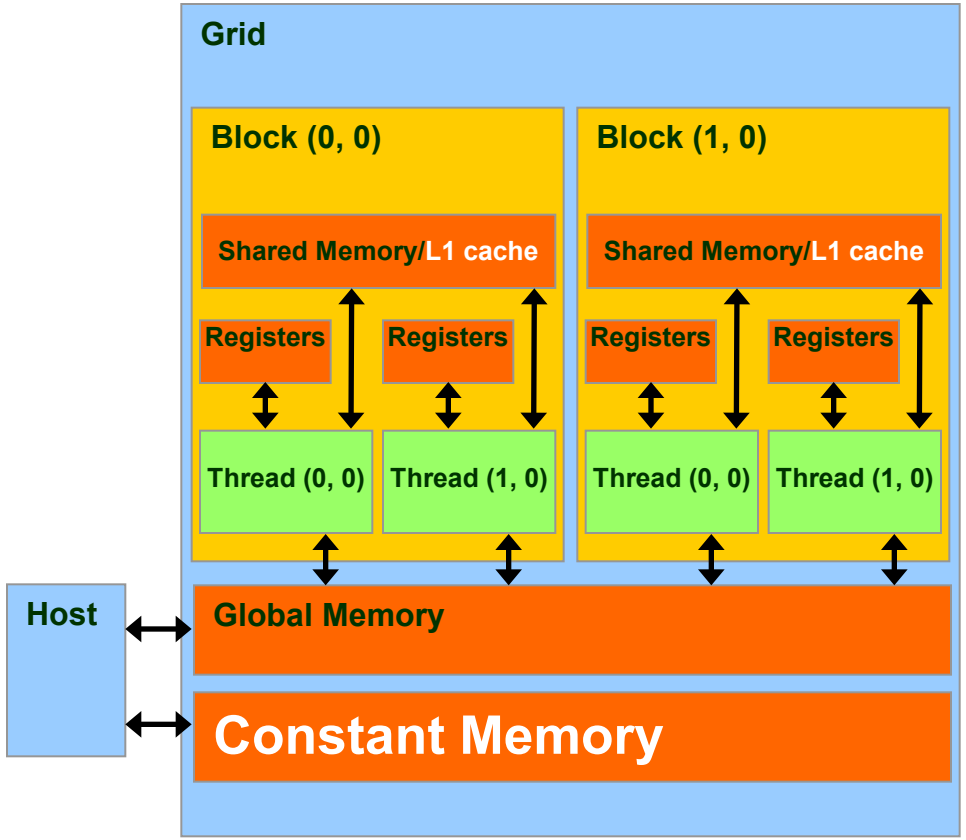
- Elements of M are called mask (kernel, filter) coefficients (weights)
  - Calculation of all output P elements needs M
  - M is not changed during grid execution

- Bonus - M elements are accessed in the same order when calculating all P elements

- M is a good candidate for Constant Memory

# Programmer View of CUDA Memories

- Each thread can:
  - Read/write per-thread **registers (~1 cycle)**
  - Read/write per-block **shared memory (~5 cycles)**
  - Read/write per-grid **global memory (~500 cycles)**
  - Read/only per-grid **constant memory (~5 cycles with caching)**



# Memory Hierarchies

- Review: If we had to go to global memory to access data all the time, the execution speed of GPUs would be limited by the global memory bandwidth
  - We saw the use of shared memory in tiled matrix multiplication to reduce this limitation

- Another important solution: Caches

# Cache

- A cache is an “array” of cache lines
  - A cache line can usually hold data from several consecutive memory addresses

- When data is requested from the global memory, an entire cache line that includes the data being accessed is loaded into the cache, in an attempt to reduce global memory requests
  - The data in the cache is a “copy” of the original data in global memory
  - Additional hardware is used to remember the addresses of the data in the cache line

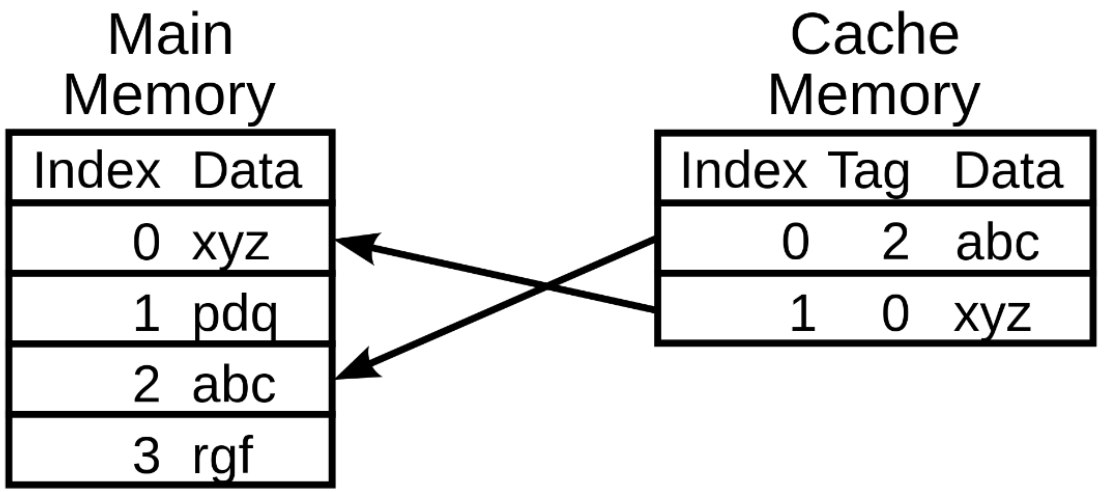
# Caches Store Lines of Memory

Recall: memory bursts

- contain around **1024 bits** (**128B**) from consecutive (linear) addresses
- Let's call a single burst a **line**

What's a **cache**?

- An **array of cache lines** (and tags)
- Memory **read produces** a **line**,
- **cache stores** a **copy** of the line, and
- tag records line's memory address



# Caches and Locality

- Some definitions:
  - Spatial locality:  when the data elements stored in consecutive memory locations are accessed consecutively
  - Temporal locality: when the same data element is accessed multiple times in short period of time

- Both spatial locality and temporal locality improve the performance of caches

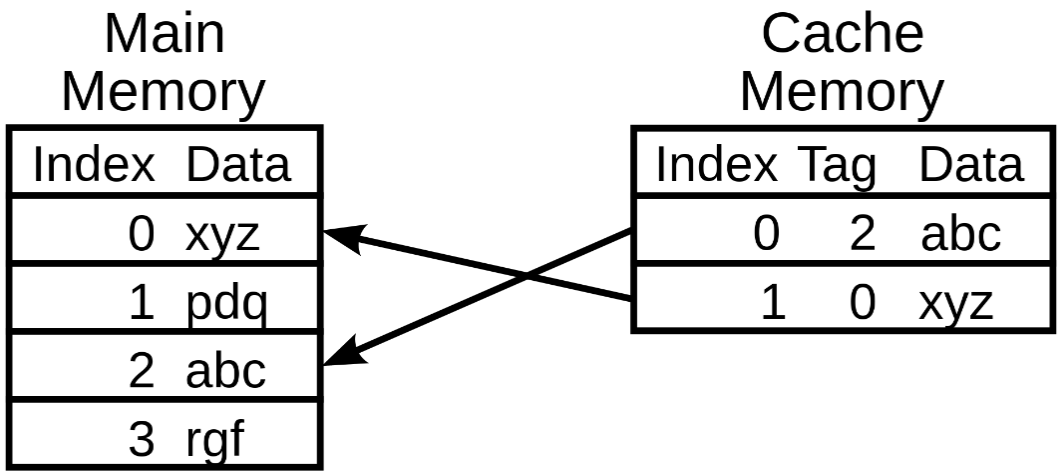
# Memory Accesses Show Locality

- An executing program
  - loads and store data from memory.
- **Consider sequence of addresses** accessed.
- **Sequence** usually **shows** both types of **locality**:
  - **spatial**: accessing **X implies** accessing **X+1** (and X+2, and so forth) **soon**
  - **temporal**: accessing **X** implies accessing **X again soon**

(Caches improve performance for both types.)

# Caches Can’t Hold Everything

- Caches are smaller than memory.
- **When cache is full**,
  - must make room for new line,
  - usually by **discard**ing **least recently used line**.



# Shared Memory vs. Cache

- Shared memory in CUDA is another type of temporary storage used to relieve main memory contention
  - In terms of distance from the SMs, shared memory is similar to L1 cache

- Unlike cache, shared memory does not necessarily hold a copy of data that is also in main memory
  - Shared mem requires explicit data transfer instructions into locations in the shared mem, whereas cache doesn't

# Shared Memory vs. Cache

- Caches vs. shared memory
  - Both on chip*, with similar performance
  - (As of Volta generation, both using the same physical resources, allocated dynamically!)
- **What's the difference?**
  - **Programmer controls shared memory** contents (called a scratchpad)
  - **Microarchitecture** automatically **determines contents of cache**.

*Static RAM, not DRAM, by the way—see ECE120/CS233.

# Constant Cache in GPUs

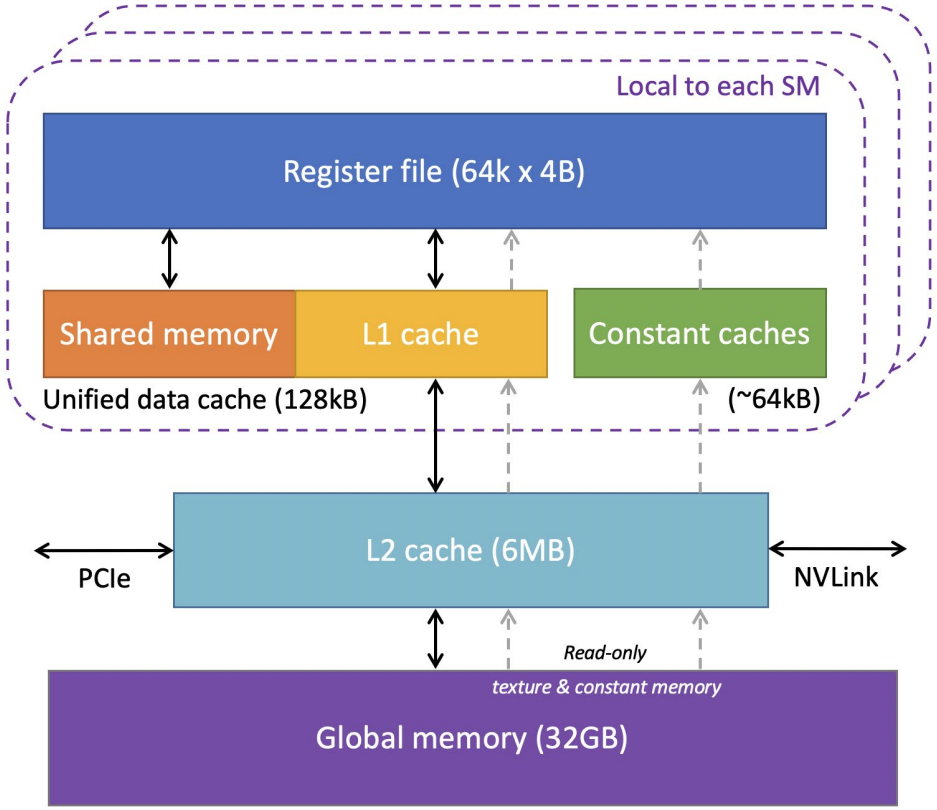
- Modification to cached data needs to be (eventually) reflected back to the original data in global memory
  - Requires logic to track the modified status, etc.

- Constant cache is a special cache for constant data that will not be modified during kernel execution by a grid
  - Data declared in the constant memory is not modified during kernel execution.
  - Constant cache can be accessed with higher throughput than L1 cache for some common patterns

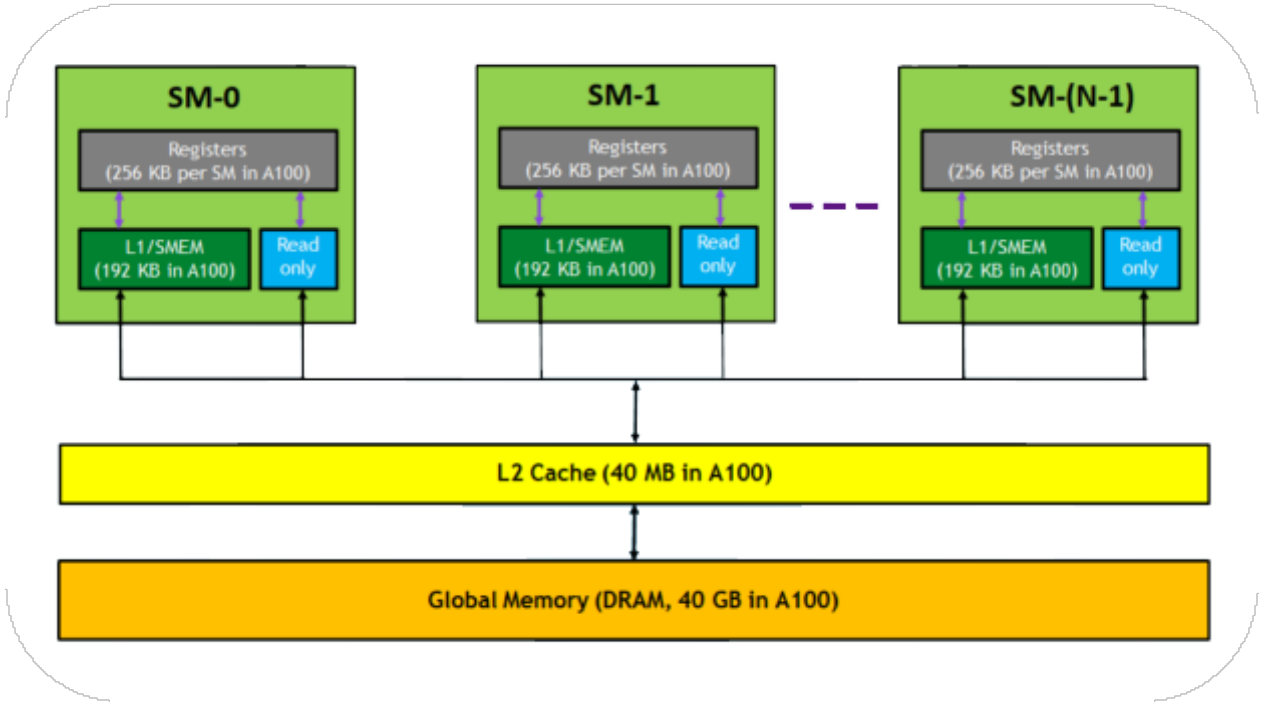
# GPU Has Constant and L1 Caches

- **To support writes** (modification of lines),
  - **changes** must be **copied back to memory**, and
  - cache must **track** modification **status.**
  - **L1 cache** in GPU (for global memory accesses) **supports writes**.
- **Cache for constant** / texture **memory**
  - Special case: **lines are read-only**
  - Enables higher-throughput access than L1 for common GPU kernel access patterns

# GPU L2/L1 Caches




**V100**




**A100**

# Using Constant Memory

- Declare constant memory array as global variable
  - Outside the CUDA kernel and any host function

```
__constant__ float filter_c[FILTER_DIM];
```

- Must initialize constant memory from the host:
  - Cannot modify during execution

```
cudaMemcpyToSymbol(filter_c, filter, FILTER_DIM * sizeof(float),
                   offset = 0, kind = cudaMemcpyHostToDevice);
```

- Can only allocate up to 64KB
  - Otherwise, input is also constant, but it is too large to put in constant memory

# Host Code Example

// MASK_WIDTH is the size of the mask

```
// global variable, outside any kernel/function
__constant__ float Mc[MASK_WIDTH];

// Initialize Mask
float Mask[MASK_WIDTH]
for(unsigned int i = 0; i < MASK_WIDTH; i++) {
    Mask[i] = (rand() / (float)RAND_MAX);
    if(rand() % 2) Mask[i] = - Mask[i];
}
cudaMemcpyToSymbol(Mc, Mask, MASK_WIDTH*sizeof(float));

ConvolutionKernel<<<dimGrid, dimBlock>>>(Nd, Pd, MASK_WIDTH, WIDTH);
```

# A 1D Convolution Kernel with Constant Memory

- This kernel forces all elements outside the valid range to 0

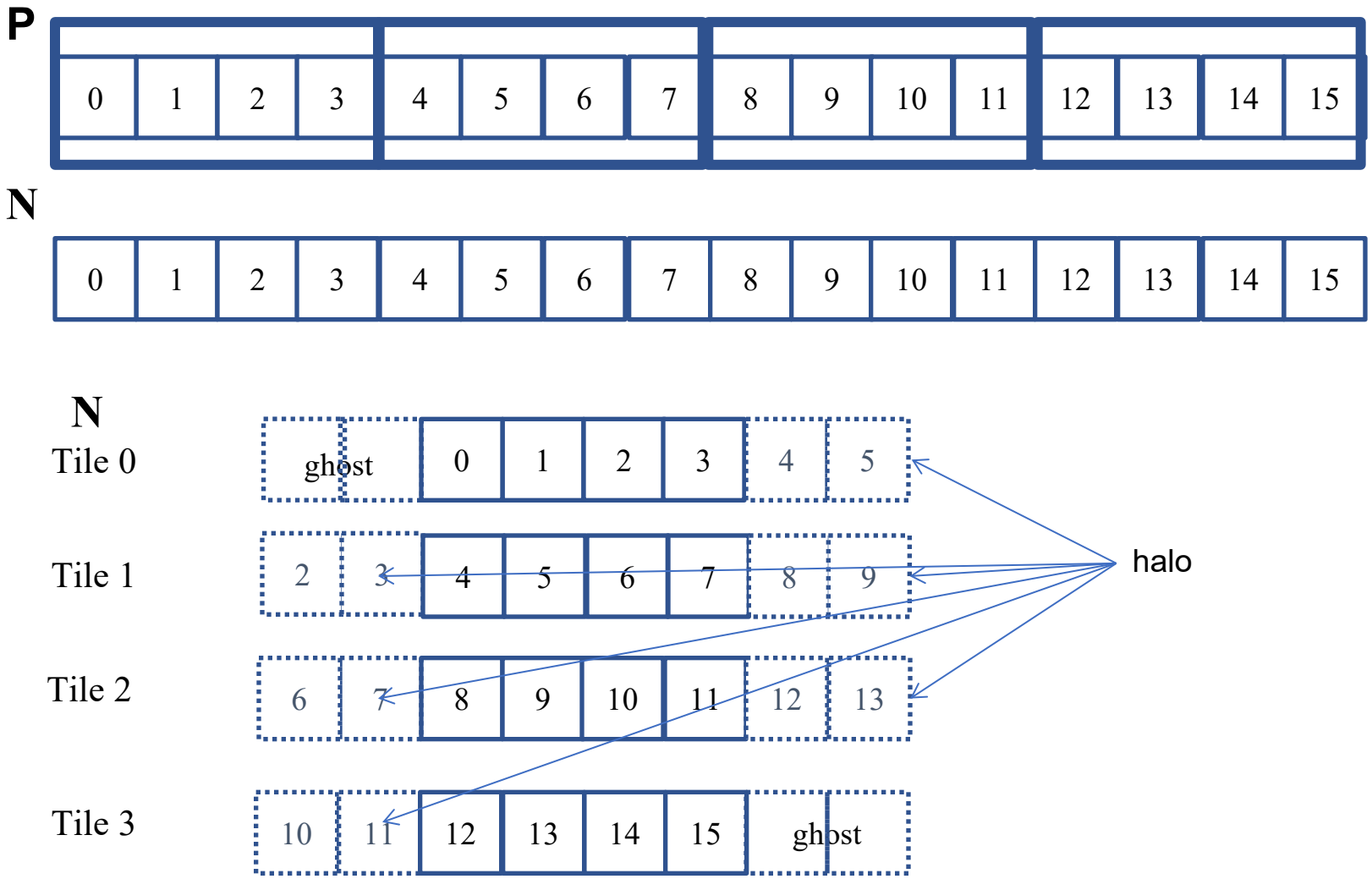
```
__global__ void
convolution_1D_basic_kernel(float *N, float *M, float *P, int Mask_Width, int Width)
{
  int i = blockIdx.x*blockDim.x + threadIdx.x;

  float Pvalue = 0;
  int N_start_point = i - (Mask_Width/2);

  for (int j = 0; j < Mask_Width; j++) {
    if (((N_start_point + j) >= 0) && ((N_start_point + j) < Width)) {
      Pvalue += N[N_start_point + j]* Mc[j];
    }
  }

  P[i] = Pvalue;
}
```

# Are We Memory Limited?

- For the 1D case, every output element requires 2*`MASK_WIDTH` loads (of M and N each) and 2*`MASK_WIDTH` floating point operations. **Memory limited.**

- For the 2D case, every output element requires 2*`MASK_WIDTH`$^2$ loads and 2*`MASK_WIDTH`$^2$ floating point operations. **Memory limited.**

# Tiled 1D Convolution Basic Idea



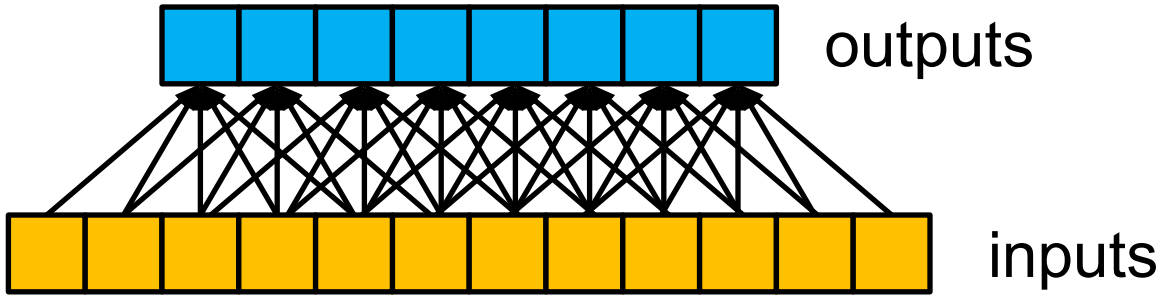
# What Shall We Parallelize?

- In other words,
  - **What should one thread do?**

- **One answer:**
  - (same as with vector sum and matrix multiply)
  - **compute an output element!**

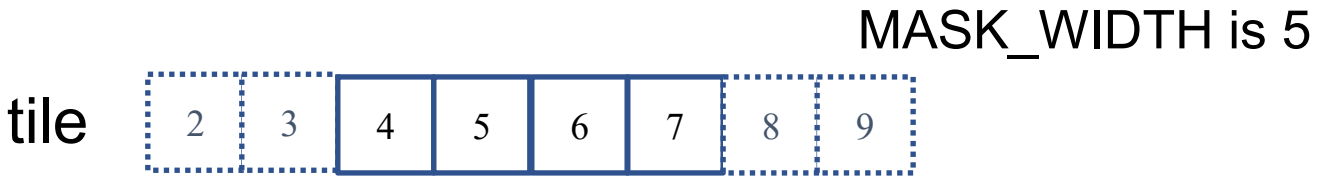
# Should We Use Shared Memory?

- In other words,
  - **Can we reuse data read from global memory?**
- Let's look at the computation again…




- Reuse reduces global memory bandwidth, so **let's use shared memory**.

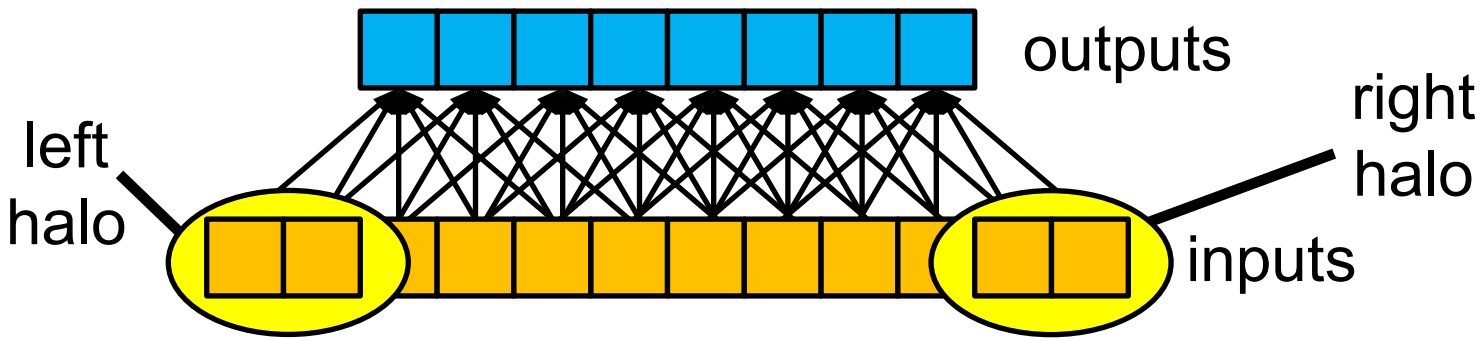
# How Much Reuse is Possible?




- Element 2 is used by thread 4 (1×)
- Element 3 is used by threads 4, 5 (2×)
- Element 4 is used by threads 4, 5, 6 (3×)
- Element 5 is used by threads 4, 5, 6, 7 (4×)
- Element 6 is used by threads 4, 5, 6, 7 (4×)
- Element 7 is used by threads 5, 6, 7 (3×)
- Element 8 is used by threads 6, 7 (2×)
- Element 9 is used by thread 7 (1×)

# What About the Halos?

- In other words,
  - **Do we also copy halos into shared memory?**




- Let's **consider both** possible answers.

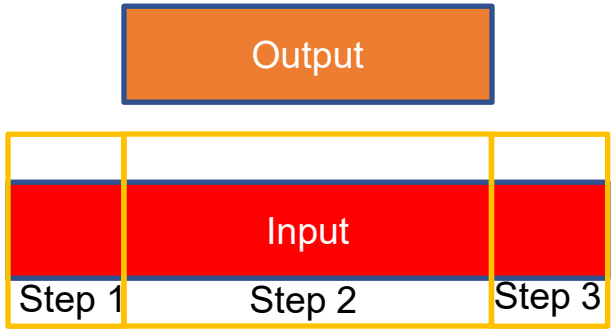
# Can Access Halo from Global Memory

- Approach:
  - threads **read halo values**
  - directly **from global memory**.
- Advantage:
  - optimize reuse of shared memory
  - (halo reuse is smaller).
- Disadvantages:
  - **Branch divergence**! (shared vs. global reads)
  - Halo **too narrow to fill** a memory **burst**

# Can Load Halo to Shared Memory

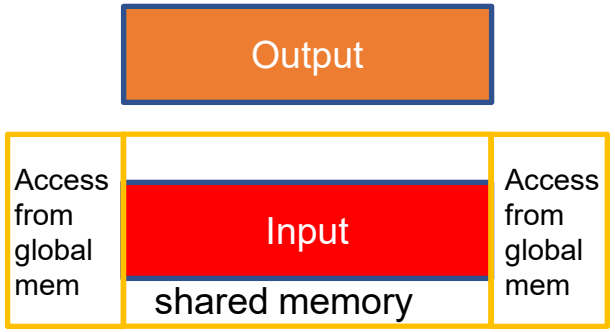
- Approach:
  - **load halos to shared memory**.
- Advantages:
  - **Coalesce global memory accesses**.
  - **No branch divergence during computation**.
- Disadvantages:
  - Some threads must do >1 load, so **some branch divergence** in reading data.
  - Slightly more shared memory needed.

# Three Tiling Strategies



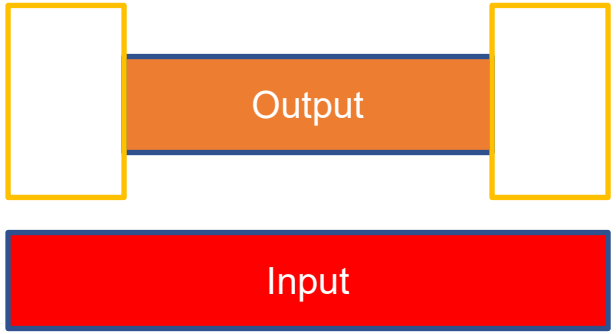

**Strategy 1**

1. Block size covers **output** tile
2. Use multiple steps to load input tile




**Strategy 3**

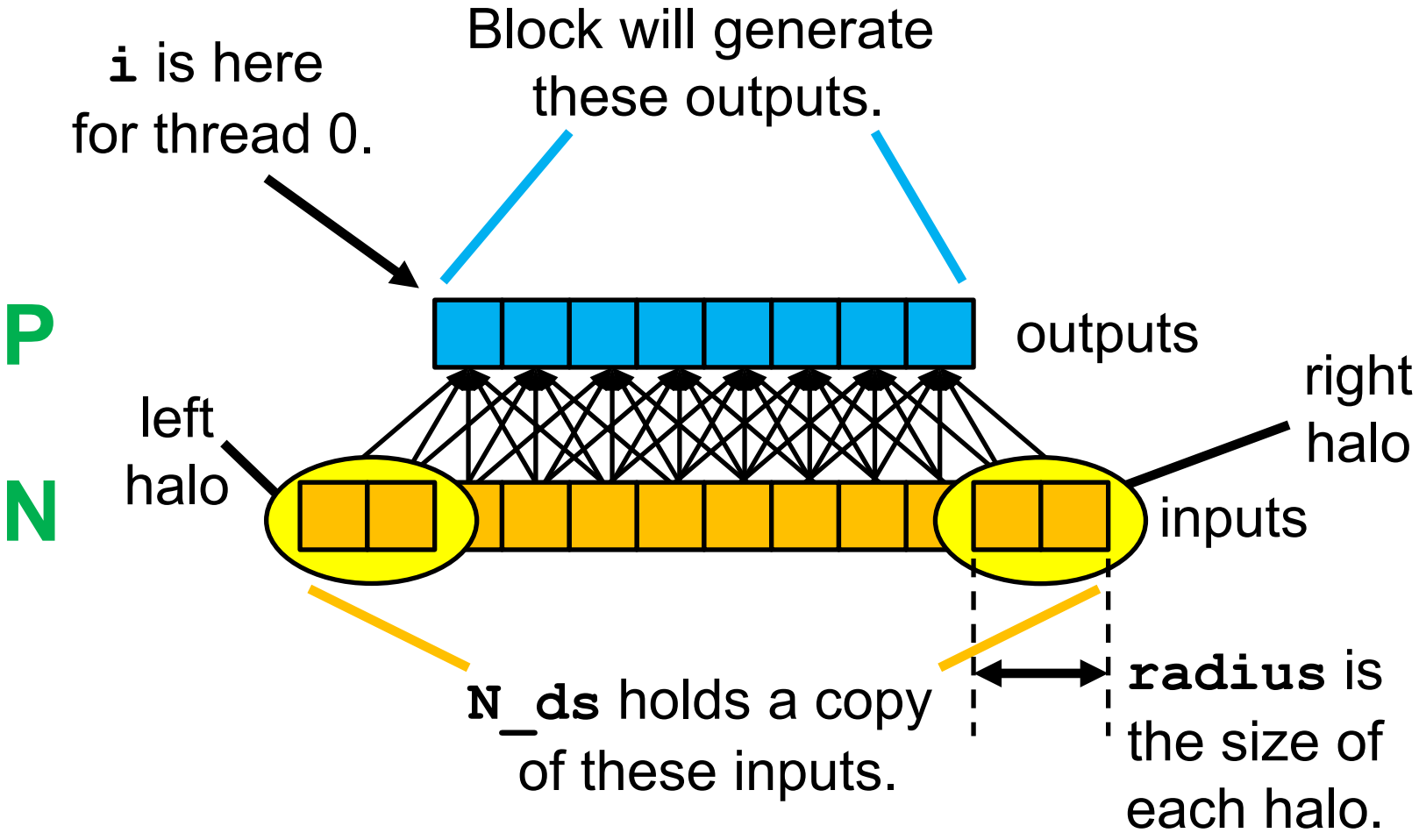
1. Block size covers **output** tile
2. Load only “core” of input tile
3. Access halo cells from global memory



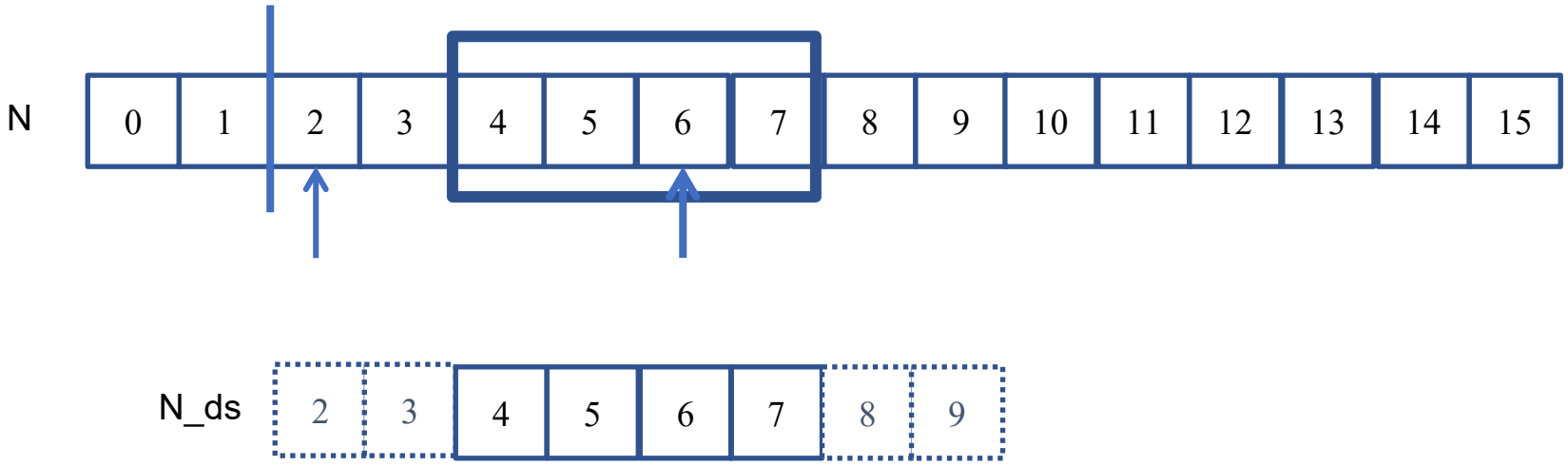

**Strategy 2**

1. Block size covers **input** tile
2. Load input tile in one step
3. Turn off some threads when calculating output

# Strategy 1: Variable Meanings for a Block

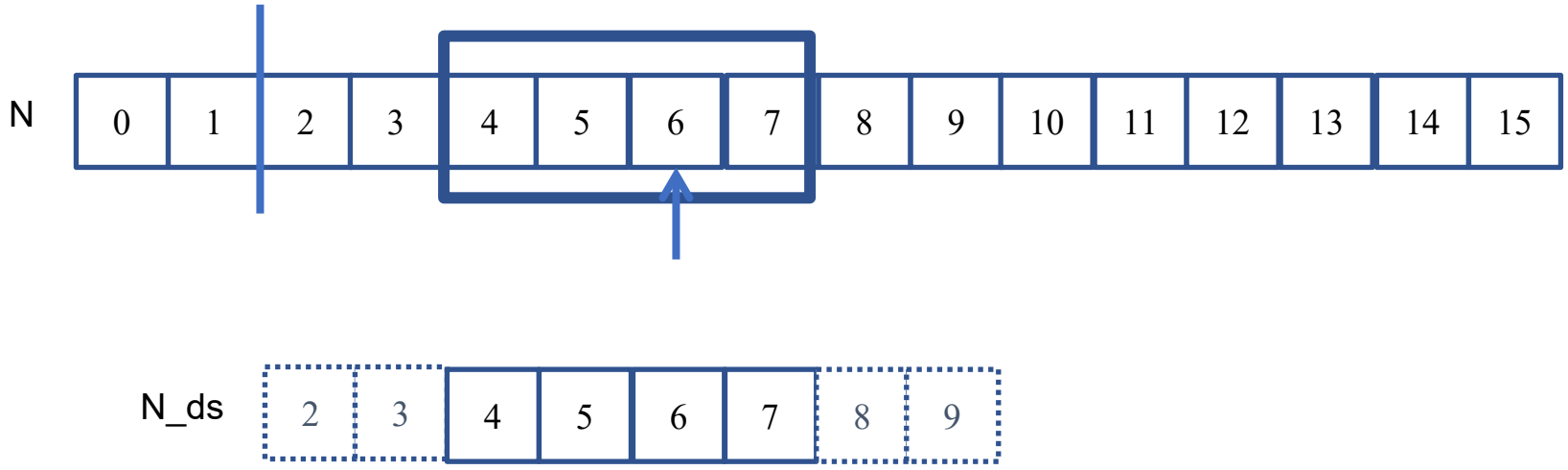


# Loading the Left Halo



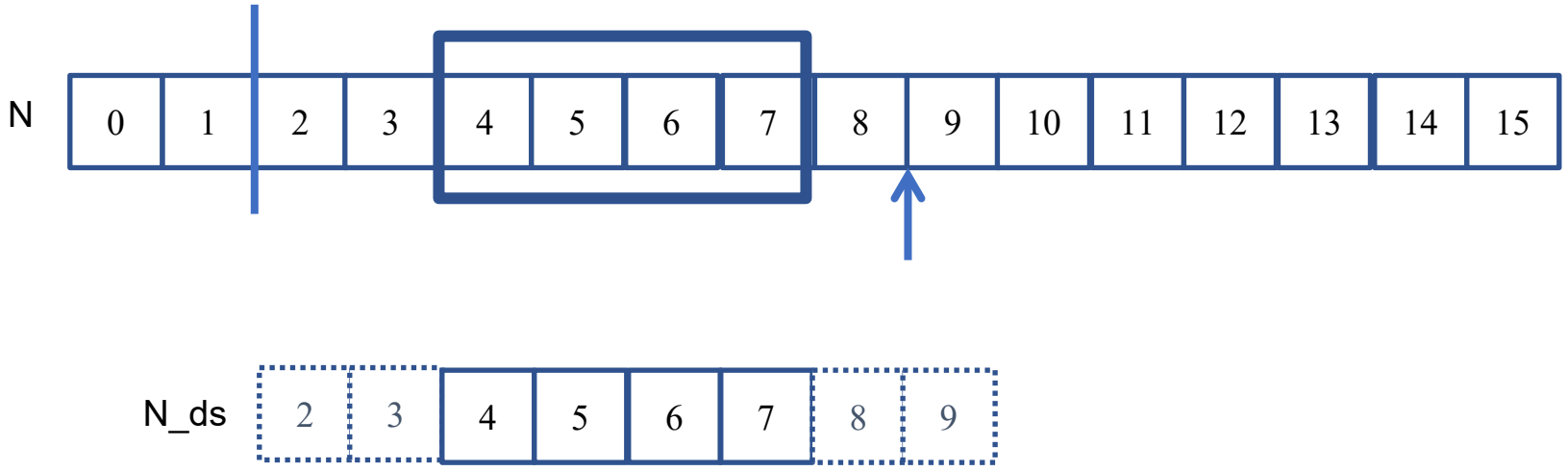

```
int radius = Mask_Width / 2;
int halo_index_left = (blockIdx.x - 1) * blockDim.x + threadIdx.x;
if (threadIdx.x >= (blockDim.x - radius)) {
   N_ds[threadIdx.x - (blockDim.x - radius)] =
      (halo_index_left < 0) ? 0 : N[halo_index_left];
}
```

# Loading the Internal Elements




```
int index = blockIdx.x * blockDim.x + threadIdx.x;
if ((blockIdx.x * blockDim.x + threadIdx.x) < Width)
  N_ds[radius + threadIdx.x] = N[index];
else
  N_ds[radius + threadIdx.x] = 0.0f;
```

# Loading the Right Halo

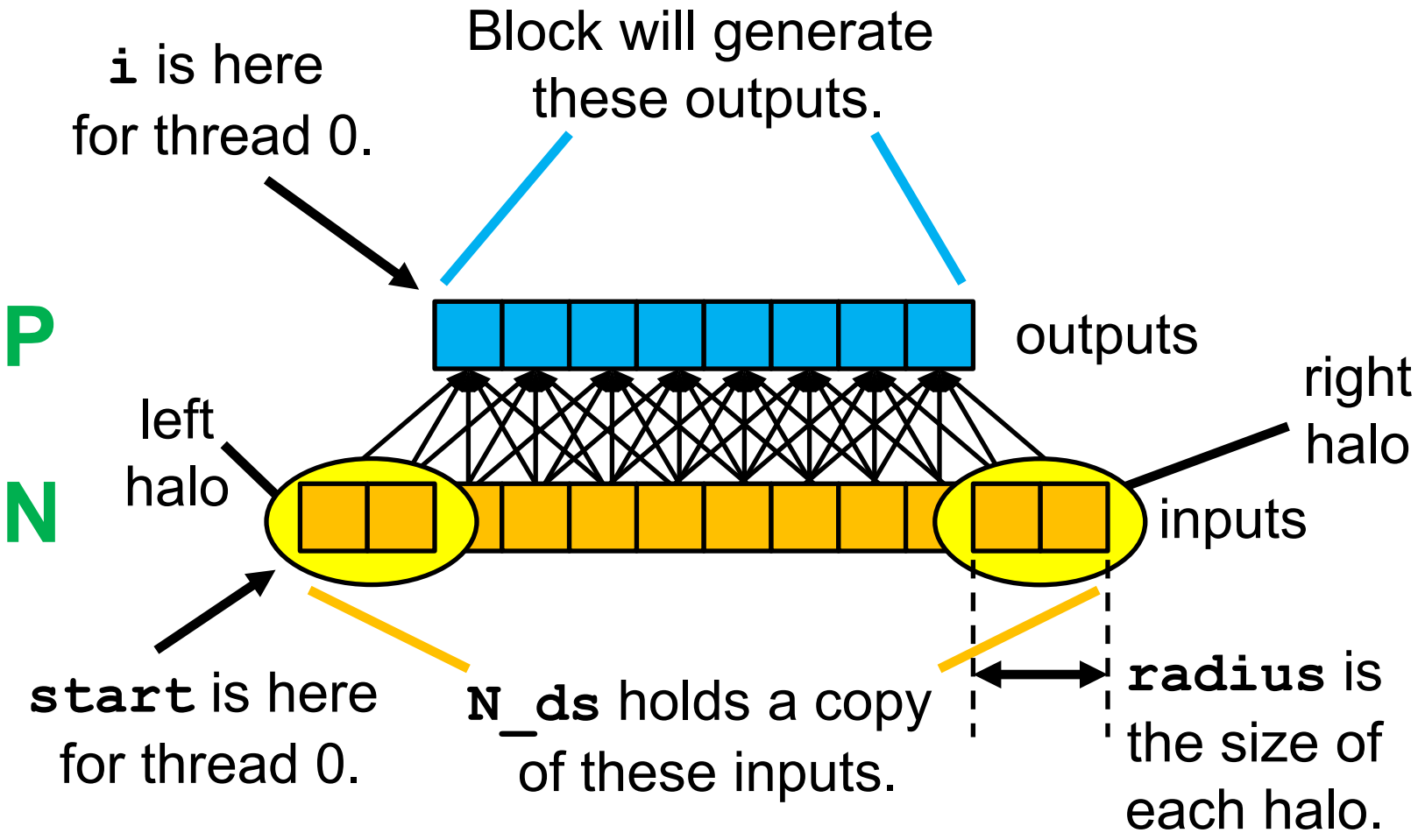



```
int halo_index_right = (blockIdx.x + 1)*blockDim.x + threadIdx.x;
if (threadIdx.x < radius) {
    N_ds[radius + blockDim.x + threadIdx.x] =
      (halo_index_right >= Width) ? 0 : N[halo_index_right];
}
```

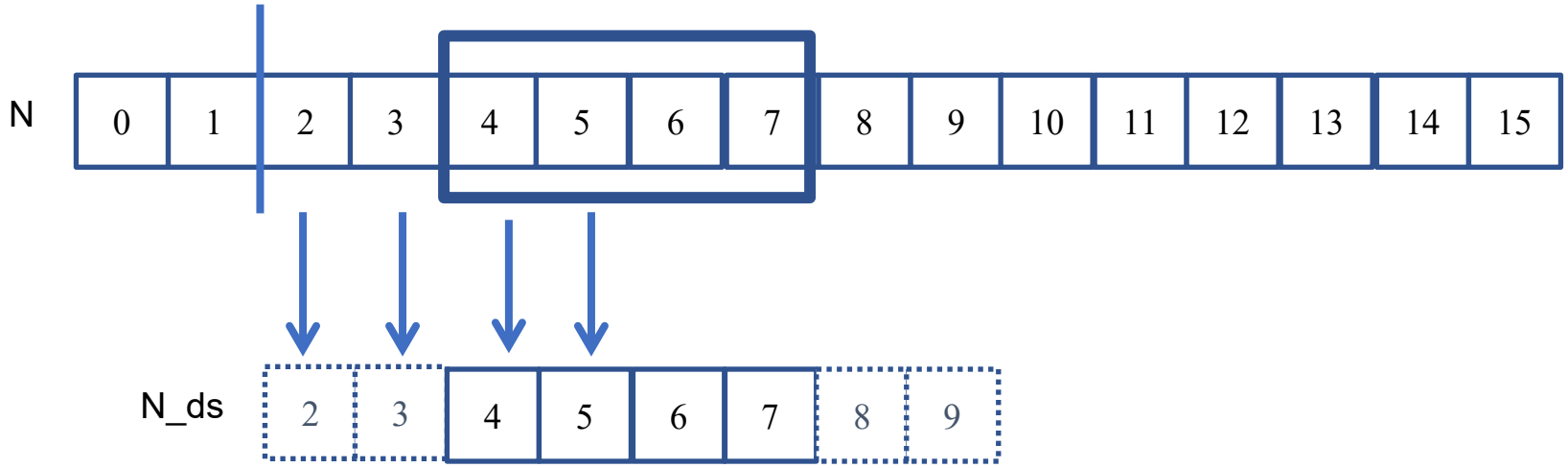
# Putting It All Together

```
// Load left halo
// Load internal elements
// Load right halo
// Compute and store results

  __syncthreads();

  float Pvalue = 0;
  for(int j = 0; j < Mask_Width; j++) {
    Pvalue += N_ds[threadIdx.x + j] * Mc[j];
  }

  P[i] = Pvalue;
```

# Alternative Implementation of Strategy 1

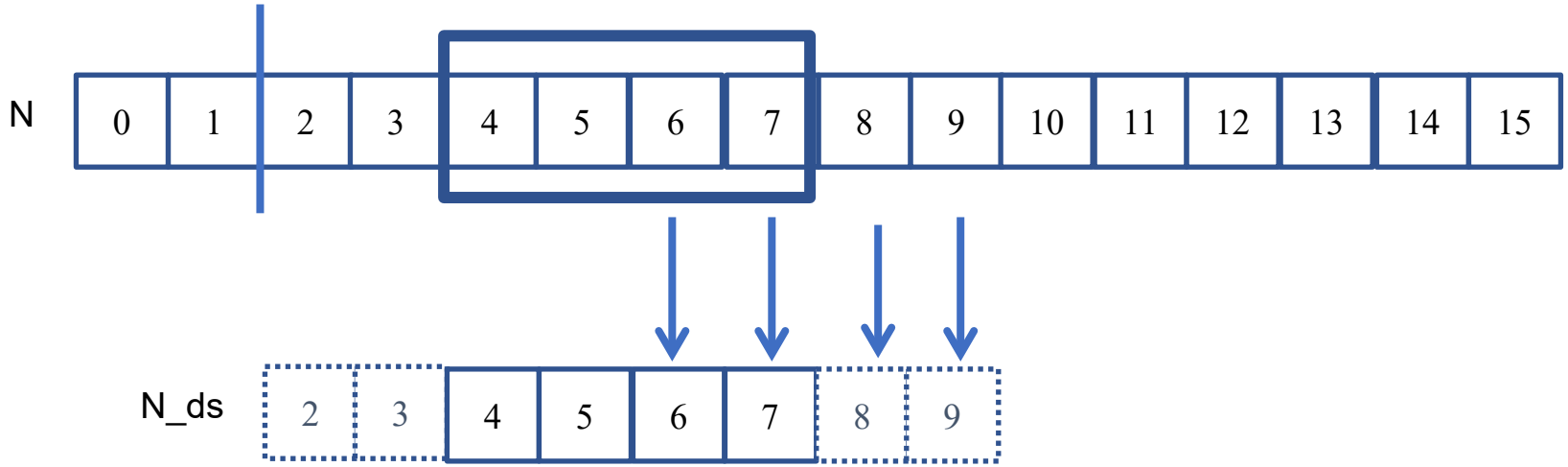


# Load the Input Data – Step 1




```
int start = i - radius;
if (0 <= start && Width > start) {     // all threads
  N_ds[threadIdx.x] = N[start];
} else {
  N_ds[threadIdx.x] = 0.0f;
}
```

# Load the Input Data – Step 2




```
if (MASK_WIDTH - 1 > threadIdx.x) {    // some threads
  start += TILE_SIZE;
  if (Width > start) {
    N_ds[threadIdx.x + TILE_SIZE] = N[start];
  } else {
    N_ds[threadIdx.x + TILE_SIZE] = 0.0f;
  }
}
```

# Strategy 3 – Loading Data

```
int i = blockIdx.x * blockDim.x + threadIdx.x;

__shared__ float  N_ds[TILE_WIDTH];

N_ds[threadIdx.x] = N[i];  // boundary checking is missing here

__syncthreads();
```

# Strategy 3 – computing

```
int radius = Mask_Width / 2;
int This_tile_start_point = blockIdx.x * blockDim.x;
int Next_tile_start_point = (blockIdx.x + 1) * blockDim.x;
int N_start_point = i - radius;

float Pvalue = 0;
for (int j = 0; j < Mask_Width; j ++) {
   int N_index = N_start_point + j;
   if (N_index >= 0  && N_index < Width) {
     if ((N_index >= This_tile_start_point) &&
         (N_index < Next_tile_start_point))
       Pvalue += N_ds[threadIdx.x-radius+j] * M[j];
     else
       Pvalue += N[N_index] * Mc[j];
   }
}
P[i] = Pvalue;
```

# Review: What Shall We Parallelize?

- In other words,
  - **What should one thread do?**

- **One answer:**
  - (same as with vector sum and matrix multiply)
  - **compute an output element!**
    - **Strategy 1 & 3**

- **Is that our only choice? (What about Strategy 2?)**

# Strategy 2: Parallelize Loading of a Tile

- Alternately,
  - **each thread loads** one input element, and
  - **some threads compute** an output.
- Advantage:
  - **No** branch **divergence for load** (high latency).
  - **Avoid narrow global access** (2 × halo width).
- Disadvantage:
  - Branch **divergence for compute** (low latency).

- Next lecture…

# Any More Questions?

Things to Read:

**Textbook chapter 7**

**CUDA BPG: Memory Optimizations**

Things to Do:

**Submit Lab 3**

**Sign up for GPT-2 project, if you wish**